Password Manager — Implementation Documentation

Exercise 1: Creating the Environment

The create_environment.sh script was run as provided to scaffold the project. The key construct is the heredoc (<< 'EOF'), which passes a multi-line string to cat and redirects it into password_manager.sh. Quoting the delimiter prevents variable substitution inside the block, which is important as the string being written contains bash code. A guard (if [! -f ...]) prevents accidental overwrites on re-runs. chmod +x grants execute permission to the main script. No deviations were made.

Exercise 2: Creating the Menu System

Two additions were made to password_manager.sh. The show_menu function displays four numbered options, reads user input, and routes it via a case statement. Option 4 calls exit 0; unrecognised input falls to the * default which prints an error. Inside main, a while true loop continuously calls show_menu, keeping the interface active until the user exits. The menu is wrapped in hyphens for pretty printing, otherwise no deviations were made.

Exercise 3: Master Password Creation

The initialization logic was implemented in src/initialize.sh and sourced in password_manager.sh. The initialize function checks for data/.MASTER and delegates to create_master_password or check_master_password accordingly.

create_master_password uses read -s to silently collect the password twice, looping until both entries match. A SHA-512 hash is then created with a random salt:

```
openssl passwd -6 -salt $(openssl rand -base64 16) "$password"
```

The hash is saved to data/.MASTER and the plaintext is kept in the global variable MASTER_PASSWORD. Because the script is sourced rather than executed in a subshell, this variable remains accessible in password_manager.sh for the rest of the session, avoiding repeated re-authentication. No deviations were made.

Exercise 4: Implementing Password Verification

check_master_password reads the stored hash from data/.MASTER, extracts the salt using the get_salt helper, then loops prompting for the master password. Each attempt is re-hashed with the same salt and compared to the stored value; the loop breaks only on a match.

MASTER_PASSWORD is set globally on success for session-wide use. No deviations were made.

Exercise 5: Creating and Storing Passwords

Three functions were added to src/passwords.sh. generate_password wraps openssl rand -base64 24 to produce a random password. encrypt_password encrypts a plaintext password

with the master password using AES-256-CBC, PBKDF2, and 64000 iterations (above the 10,000 minimum) to resist brute-force:

```
echo "$2" | openssl enc -aes-256-cbc -pbkdf2 -iter 64000 -a -k "$1"
```

new_password drives the creation flow: it accepts 'q' to quit, prompts for an account name, warns if a file already exists, asks the user to confirm the name, then calls generate_password and encrypt_password and writes the ciphertext to data/passwords/${account_name}. A small deviation from the specs: if a file with an account name already exists, the user is asked if they want to overwrite it.

Exercise 6: Password Retrieval

decrypt_password reverses the encryption using the same cipher, KDF, and iteration count with the -d flag. retrieve_password checks whether data/passwords/ is empty (returning 1 if so), then loops prompting for an account name until a matching file is found. The encrypted content is decrypted and passed to display_password.

As an expansion on the spec, display_password clears the terminal after the user presses Enter, preventing the plaintext password from lingering on screen.

Exercise 7: Listing Accounts

list_accounts was implemented in src/utils.sh. It checks whether data/passwords/ is empty using ls -A and prints a message if so (returning 1). Otherwise, ls -1 data/passwords lists each account name on its own line. The function is sourced in password_manager.sh and called from the menu's case statement. No deviations were made.

General Notes

All source files are sourced via relative paths at the top of password_manager.sh, which requires the script to be run from the project root. MASTER_PASSWORD is held in memory for the session and never written to disk, reducing the risk of plaintext exposure after the program terminates (though the risk is not entirely eliminated, because the master password can potentially be written to swap memory). Storing MASTER_PASSWORD in a global variable ensures that the user needs to be authenticated only once per session.